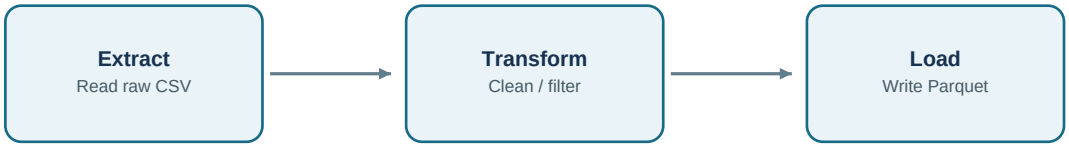
CLOUD DATA ENGINEERING LABORATORY PRACTICALS

Experiments 1 to 10



Practical Record & Exam Preparation Notes

Platforms: Amazon S3 • Databricks • Apache Spark • PySpark • Spark SQL

Name: _____

Class / Division: _____

Roll No.: _____

Academic Year: _____

Contents

1. Experiment 1 – Cloud Storage

2. Experiment 2 – Databricks Workspace and Cluster

3. Experiment 3 – Databricks and Cloud Storage Integration

4. Experiment 4 – Loading CSV and JSON Data into DataFrames

5. Experiment 5 – Data Preprocessing Using PySpark

6. Experiment 6 – SQL Operations in Databricks

7. Experiment 7 – Data Partitioning and Partition Pruning

8. Experiment 8 – ETL Pipeline Using Databricks

9. Experiment 9 – Real-Time Data Streaming

10. Experiment 10 – Sales Analytics Dashboard

How to use this record

Each experiment contains the aim, a short introduction, procedure, sample code, expected output, result, and practical notes. Replace placeholder cloud paths and resource names with values from your own account. The diagrams are illustrative schematics, not screenshots of a live AWS or Databricks console.

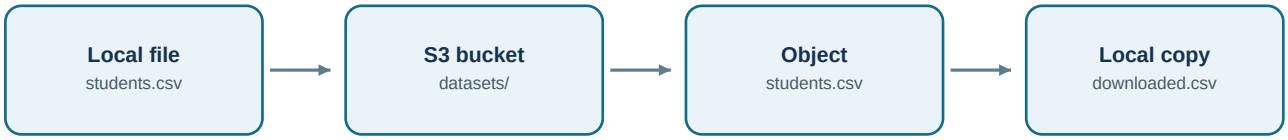
Experiment 1 – Cloud Storage

Aim: Create a cloud storage bucket and upload a dataset (CSV/JSON). Demonstrate basic operations such as upload, download, and file organization using Amazon S3/Azure Blob Storage.

Platform Used: AWS Amazon S3 (Azure Blob Storage is an alternative)

1. Introduction

Amazon S3 is an object storage service. Files are stored as objects inside buckets. A bucket belongs to a selected AWS Region. S3 folders are logical prefixes in object keys.



2. Procedure

Step 1: Create a bucket — Open the AWS Management Console, search for S3, and choose Create bucket. Enter a globally unique name such as **student-cloud-practical-2026-123**. Select Asia Pacific (Mumbai), keep Block Public Access enabled, and create the bucket.

Step 2: Create a CSV file — Create **students.csv** with the sample data shown below.

Step 3: Upload the file — Open the bucket, choose Upload, add students.csv, and select Upload.

Step 4: Organize files — Create a folder/prefix named **datasets**. Upload or move students.csv into that folder.

Step 5: Download and verify — Select the object and choose Download. Open the downloaded file and compare its contents with the original.

3. Program / Commands

Sample CSV

```
Student_ID,Name,Department,Marks
101,Ajay,CSE,85
102,Rahul,IT,90
103,Priya,CSE,88
104,Amit,ENTC,78
105,Neha,CSE,92
```

Optional AWS CLI commands

```
aws configure
aws s3 mb s3://my-cloud-practical-2026 --region ap-south-1
aws s3 cp students.csv s3://my-cloud-practical-2026/datasets/students.csv
aws s3 ls s3://my-cloud-practical-2026/datasets/
aws s3 cp s3://my-cloud-practical-2026/datasets/students.csv downloaded_students.csv
```

4. Expected Output

Bucket created; CSV uploaded; datasets prefix created; file downloaded and verified.

5. Result

Successfully created a cloud storage bucket, uploaded a CSV dataset, organized the object using a folder-like prefix, and downloaded the file.

6. Practical Notes

Keep public access blocked. Bucket names must be globally unique. AWS CLI commands require configured credentials and suitable S3 permissions.

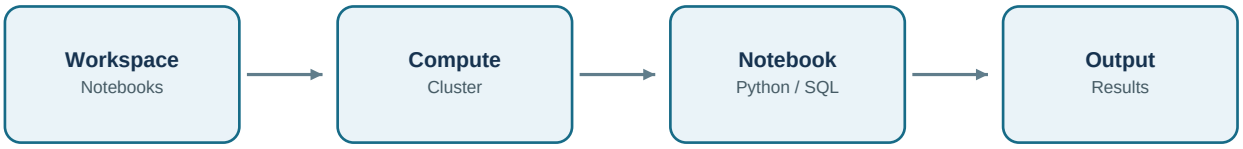
Experiment 2 – Databricks Workspace and Cluster

Aim: Set up a workspace and create a cluster in Databricks. Verify cluster configuration and execute a simple notebook command.

Platform Used: Databricks workspace and available compute

1. Introduction

A Databricks workspace is an environment for notebooks, jobs, data, and other assets. Compute resources execute notebook code. Depending on the Databricks edition, compute may be a manually configured cluster or managed serverless compute.



2. Procedure

Step 1: Open Databricks — Sign in to Databricks and open the workspace available to your account.

Step 2: Create or access a workspace — If your account provides the account console and workspace creation permission, create a workspace and select the available cloud and region. Otherwise, use the workspace provisioned for your account.

Step 3: Create compute — Open Compute. If manual cluster creation is available, choose Create compute, enter **My-Practical-Cluster**, select a runtime and node type, and create it. If only serverless compute is available, use the provided serverless compute.

Step 4: Verify configuration — Check the compute status, runtime version, node type, and worker configuration where those settings are exposed.

Step 5: Create a notebook — Create a Python notebook named **Cloud_Practical_Experiment_2** and attach the available compute.

3. Program / Commands

Simple notebook command

```
print("Hello, Databricks!")
print("My first Databricks practical")

a = 10
b = 20
print("Addition:", a + b)
```

4. Expected Output

The notebook displays Hello, Databricks!, My first Databricks practical, and Addition: 30.

5. Result

Successfully accessed a Databricks workspace, used available compute, verified its configuration, and executed Python commands in a notebook.

6. Practical Notes

Databricks Free Edition may provide managed serverless compute rather than allowing users to create a traditional cluster. Report the compute type actually available; do not claim to have created a manual cluster if you did not.

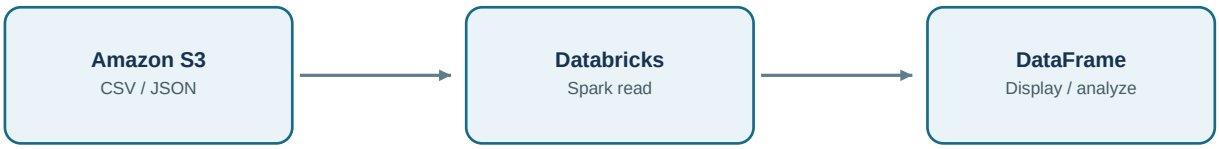
Experiment 3 – Databricks and Cloud Storage Integration

Aim: Establish a connection between Databricks and Amazon S3/Azure Blob Storage, access files stored in cloud storage, and display the stored data in Databricks.

Platform Used: Databricks and Amazon S3

1. Introduction

Cloud storage integration allows Databricks to read data stored outside the workspace. Access must be configured securely through an approved cloud identity, role, credential, or external location.



2. Procedure

Step 1: Prepare cloud storage — Create an S3 bucket and upload students.csv to a datasets/ prefix.

Step 2: Configure secure access — Configure an IAM role or supported Databricks storage credential/external location with read permission to the required bucket or path. Do not place long-lived secret keys directly in notebook code.

Step 3: Create a notebook — Create a Python notebook named **S3_Integration_Practical** and attach available compute.

Step 4: Read the CSV — Use the cloud URI after access has been configured. The example below uses an S3A URI; your workspace may require a Unity Catalog external location or another supported access method.

Step 5: Display and analyze — Display the DataFrame and calculate the average marks.

3. Program / Commands

Read CSV from S3

```
file_path = "s3a://my-cloud-practical-2026/datasets/students.csv"

df = (spark.read
      .option("header", "true")
      .option("inferSchema", "true")
      .csv(file_path))

display(df)
df.show()
```

Calculate average marks

```
from pyspark.sql.functions import avg

df.select(avg("Marks").alias("Average_Marks")).show()
```

4. Expected Output

The student rows are displayed in Databricks. The average marks for 85, 90, 88, 78, and 92 is 86.6.

5. Result

Successfully configured access to cloud storage, read a CSV dataset into Databricks, displayed the data, and performed a basic aggregation.

6. Practical Notes

Replace the bucket name and path with your actual location. A bucket URI alone does not grant access. Direct S3A access may not be supported in every Databricks edition or configuration.

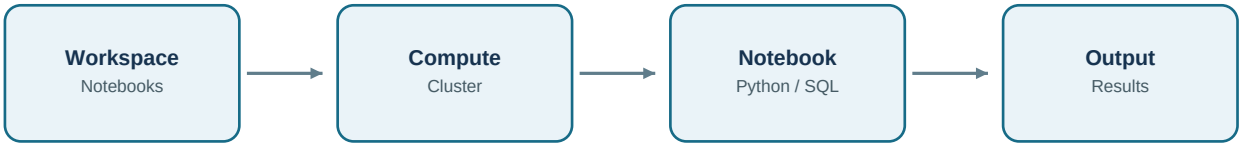
Experiment 4 – Loading CSV and JSON Data into DataFrames

Aim: Load structured and semi-structured data (CSV/JSON) into DataFrames using Databricks and display schema and sample records.

Platform Used: Databricks, Python, and PySpark

1. Introduction

A DataFrame is a distributed collection of data organized into named columns. CSV is commonly used for tabular structured data. JSON is a semi-structured format that represents data using objects, arrays, and key-value pairs.



2. Procedure

Step 1: Create a notebook — Create a Python notebook named **DataFrame_Practical**.

Step 2: Prepare datasets — Create students.csv and students.json using the sample records below. Upload them to a Databricks-accessible location.

Step 3: Load CSV — Read the CSV file with header and schema inference enabled.

Step 4: Inspect CSV — Use printSchema() and show(5) to inspect column types and sample records.

Step 5: Load JSON — Read the JSON array using multiLine=true.

Step 6: Inspect JSON — Display the JSON DataFrame, its schema, and sample records.

3. Program / Commands

CSV sample

```
Student_ID,Name,Department,Marks
101,Ajay,CSE,85
102,Rahul,IT,90
103,Priya,CSE,88
104,Amit,ENTC,78
105,Neha,CSE,92
```

Read CSV

```
csv_path = "/Volumes/<catalog>/<schema>/<volume>/students.csv"

csv_df = (spark.read
          .option("header", "true")
          .option("inferSchema", "true")
          .csv(csv_path))

csv_df.printSchema()
csv_df.show(5)
```

JSON sample

```
[
  {"Student_ID": 101, "Name": "Ajay", "Department": "CSE", "Marks": 85},
  {"Student_ID": 102, "Name": "Rahul", "Department": "IT", "Marks": 90},
  {"Student_ID": 103, "Name": "Priya", "Department": "CSE", "Marks": 88}
]
```

Read JSON

```
json_path = "/Volumes/<catalog>/<schema>/<volume>/students.json"
```

```
json_df = (spark.read
           .option("multiLine", "true")
           .json(json_path))

display(json_df)
json_df.printSchema()
json_df.show(5)
```

4. Expected Output

Both DataFrames display student records. CSV schema inference typically identifies Student_ID and Marks as integers and text columns as strings. JSON numeric types may be inferred as long integers.

5. Result

Successfully loaded structured CSV and semi-structured JSON datasets into PySpark DataFrames and displayed their schemas and sample records.

6. Practical Notes

Replace the example Volume paths with the real file paths in your workspace. The JSON example is a multi-line JSON array, so multiLine=true is used.

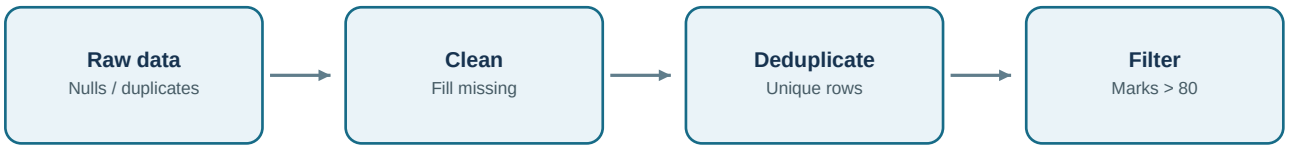
Experiment 5 – Data Preprocessing Using PySpark

Aim: Perform data preprocessing on a student dataset using PySpark in Databricks by identifying missing values, handling them appropriately, removing duplicate records, and filtering records according to a specified condition.

Platform Used: Databricks and PySpark

1. Introduction

Data preprocessing cleans raw data before analysis. Common tasks include detecting null values, replacing or removing missing values, removing duplicate rows, and filtering records.



2. Procedure

Step 1: Create a notebook and dataset — Create a Python notebook named **Data_Preprocessing** and create the sample DataFrame.

Step 2: Identify missing values — Count null values in each column using isNull().

Step 3: Handle missing values — Calculate the average of non-null Marks and fill missing Marks with the integer value of that average.

Step 4: Remove duplicates — Remove duplicate rows using Student_ID as the unique identifier.

Step 5: Filter records — Select students whose Marks are greater than 80.

Step 6: Display final data — Display the cleaned and filtered DataFrame.

3. Program / Commands

Create dataset

```
data = [
    (101, "Ajay", "CSE", 85),
    (102, "Rahul", "IT", 90),
    (103, "Priya", "CSE", None),
    (104, "Amit", "ENTC", 78),
    (105, "Neha", "CSE", 92),
    (106, "Rohan", "IT", None),
    (101, "Ajay", "CSE", 85)
]
columns = ["Student_ID", "Name", "Department", "Marks"]
df = spark.createDataFrame(data, columns)
display(df)
```

Identify nulls, fill, deduplicate, and filter

```
from pyspark.sql.functions import col, sum as spark_sum, avg

df.select([
    spark_sum(col(c).isNull().cast("int")).alias(c)
    for c in df.columns
]).show()

average_marks = df.select(avg("Marks")).first()[0]
df = df.fillna({"Marks": int(average_marks)})
df = df.dropDuplicates(["Student_ID"])

filtered_df = df.filter(col("Marks") > 80)
display(filtered_df)
filtered_df.show()
print("Total records:", filtered_df.count())
```

4. Expected Output

Two missing Marks values are identified. The average of the five non-null marks is 86.6; int(86.6) gives 86. After filling, duplicate Student_ID 101 is removed. Filtering Marks > 80 returns Ajay, Rahul, Priya, Neha, and Rohan.

5. Result

Successfully identified and handled missing values, removed duplicate student records, and filtered the cleaned dataset using PySpark.

6. Practical Notes

The average is 86.6, converted to 86 by int(). dropDuplicates(['Student_ID']) is appropriate only if Student_ID is intended to uniquely identify a student.

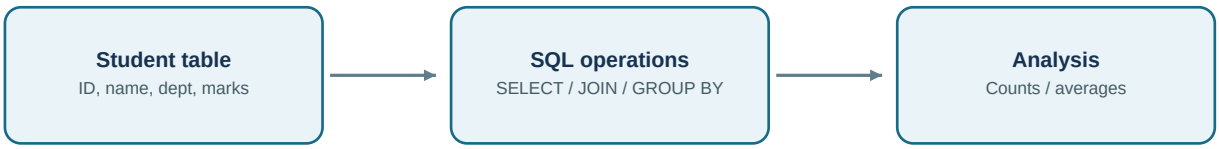
Experiment 6 – SQL Operations in Databricks

Aim: Use SQL queries in Databricks to perform operations like SELECT, JOIN, GROUP BY, and aggregation on a dataset.

Platform Used: Databricks SQL and PySpark

1. Introduction

Databricks supports Spark SQL for querying and analyzing data. SELECT retrieves records, JOIN combines related tables, GROUP BY groups records, and aggregate functions summarize values.



2. Procedure

Step 1: Create student data — Create a DataFrame containing Student_ID, Name, Department, and Marks.

Step 2: Register a temporary view — Use createOrReplaceTempView() to make the DataFrame queryable using SQL.

Step 3: Create department data — Create a second DataFrame with department codes and names, then register it as a temporary view.

Step 4: Run SELECT — Retrieve all records and filter students using WHERE.

Step 5: Run JOIN — Join the student and department views using Department.

Step 6: Run GROUP BY and aggregation — Count students by department and calculate average, maximum, minimum, sum, and count of marks.

3. Program / Commands

Create temporary views

```
students = [
    (101, "Ajay", "CSE", 85),
    (102, "Rahul", "IT", 90),
    (103, "Priya", "CSE", 88),
    (104, "Amit", "ENTC", 78),
    (105, "Neha", "CSE", 92)
]
students_df = spark.createDataFrame(
    students, ["Student_ID", "Name", "Department", "Marks"]
)
students_df.createOrReplaceTempView("students")

departments = [
    ("CSE", "Computer Science"),
    ("IT", "Information Technology"),
    ("ENTC", "Electronics")
]
departments_df = spark.createDataFrame(
    departments, ["Department", "Department_Name"]
)
departments_df.createOrReplaceTempView("departments")
```

SELECT

```
%sql
SELECT * FROM students;

%sql
SELECT * FROM students WHERE Marks > 80;
```

INNER JOIN

```
%sql
SELECT s.Student_ID, s.Name, d.Department_Name, s.Marks
FROM students s
INNER JOIN departments d
ON s.Department = d.Department;
```

GROUP BY and aggregation

```
%sql
SELECT Department, COUNT(*) AS Total_Students
FROM students
GROUP BY Department;

%sql
SELECT AVG(Marks) AS Average_Marks,
       MAX(Marks) AS Maximum_Marks,
       MIN(Marks) AS Minimum_Marks,
       SUM(Marks) AS Total_Marks,
       COUNT(*) AS Total_Students
FROM students;
```

4. Expected Output

GROUP BY returns CSE = 3 students, IT = 1, and ENTC = 1. Aggregation returns average marks = 86.6, maximum = 92, minimum = 78, total = 433, and count = 5.

5. Result

Successfully performed SELECT, WHERE, INNER JOIN, GROUP BY, and aggregation operations on student and department datasets using Databricks SQL.

6. Practical Notes

Run the Python cells that create the temporary views before running SQL cells. Temporary views are available within the same Spark session.

Experiment 7 – Data Partitioning and Partition Pruning

Aim: Store processed student data in a partitioned table using Databricks and compare query execution behavior with a non-partitioned table to understand the performance benefits of partition pruning.

Platform Used: Databricks, Delta Lake, and Spark SQL

1. Introduction

Partitioning divides a table's data into separate storage partitions based on one or more columns. Partition pruning allows Spark to skip partitions that do not match a query filter, reducing the data that needs to be scanned.



2. Procedure

Step 1: Create a student dataset — Create a DataFrame containing student details and department values.

Step 2: Create a non-partitioned table — Save the DataFrame as a Delta table without specifying a partition column.

Step 3: Create a partitioned table — Save the same DataFrame as a Delta table partitioned by Department.

Step 4: Verify tables — Use SHOW TABLES and SELECT queries to verify both tables.

Step 5: Run equivalent filters — Query both tables using WHERE Department = 'CSE'.

Step 6: Compare execution plans — Use EXPLAIN FORMATTED and inspect scan details and partition filters.

3. Program / Commands

Create dataset and tables

```
data = [
    (101, "Ajay", "CSE", 85),
    (102, "Rahul", "IT", 90),
    (103, "Priya", "CSE", 88),
    (104, "Amit", "ENTC", 78),
    (105, "Neha", "CSE", 92),
    (106, "Rohan", "IT", 86),
    (107, "Sneha", "ENTC", 89),
    (108, "Karan", "CSE", 95)
]
df = spark.createDataFrame(
    data, ["Student_ID", "Name", "Department", "Marks"]
)

df.write.format("delta").mode("overwrite") \
  .saveAsTable("students_non_partitioned")

df.write.format("delta").mode("overwrite") \
  .partitionBy("Department") \
  .saveAsTable("students_partitioned")
```

Compare query plans

```
%sql
EXPLAIN FORMATTED
SELECT * FROM students_non_partitioned
WHERE Department = 'CSE';

%sql
EXPLAIN FORMATTED
SELECT * FROM students_partitioned
WHERE Department = 'CSE';
```

4. Expected Output

The CSE query returns four records. The partitioned table's plan can show a partition filter for Department = CSE, allowing irrelevant department partitions to be skipped.

5. Result

Successfully created partitioned and non-partitioned Delta tables, executed equivalent queries, and examined execution plans to understand partition pruning.

6. Practical Notes

Eight records are too few to demonstrate a reliable timing improvement. Focus on the execution plan and partition filters. Partitioning is most useful for sufficiently large datasets and selective filters on the partition column. Databricks may recommend liquid clustering or other layouts for some workloads.

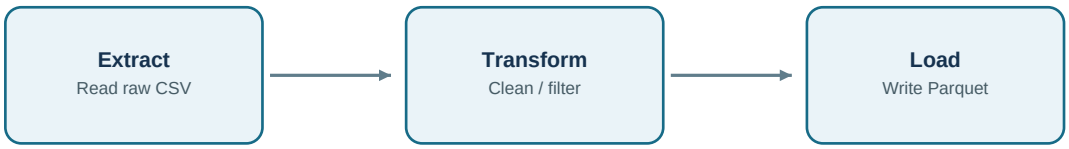
Experiment 8 – ETL Pipeline Using Databricks

Aim: Design and implement an ETL pipeline to extract data from cloud storage, transform it, and load it back into storage using Databricks or Airflow or Azure Data Factory.

Platform Used: Databricks, PySpark, and cloud storage

1. Introduction

ETL stands for Extract, Transform, and Load. Extract reads source data, Transform cleans or modifies it, and Load writes the prepared data to a destination for later use.



2. Procedure

Step 1: Prepare source data — Create students.csv and upload it to a raw/ folder in cloud storage.

Step 2: Extract — Read the CSV file into a PySpark DataFrame.

Step 3: Transform — Calculate an average for missing marks, fill missing values, remove duplicate student IDs, and filter students scoring more than 80.

Step 4: Load — Write the transformed dataset to a processed/ destination in Parquet format.

Step 5: Verify — Read the output Parquet files and display the final records.

3. Program / Commands

Extract data

```
input_path = "s3a://my-cloud-practical-2026/raw/students.csv"

df = (spark.read
      .option("header", "true")
      .option("inferSchema", "true")
      .csv(input_path))

display(df)
```

Transform data

```
from pyspark.sql.functions import col, avg

average_marks = df.select(avg("Marks")).first()[0]
df = df.fillna({"Marks": int(average_marks)})
df = df.dropDuplicates(["Student_ID"])
df = df.filter(col("Marks") > 80)

display(df)
```

Load and verify

```
output_path = "s3a://my-cloud-practical-2026/processed/"

df.write.mode("overwrite").parquet(output_path)

final_df = spark.read.parquet(output_path)
display(final_df)
final_df.show()
```

4. Expected Output

The pipeline reads raw CSV data, handles the missing mark, removes duplicate student IDs, filters marks greater than 80, and writes the output in Parquet format.

5. Result

Successfully designed and implemented an ETL pipeline using Databricks and PySpark to extract, transform, and load student data between cloud storage locations.

6. Practical Notes

Replace the bucket and paths with actual locations and configure secure storage permissions first. With the sample marks 85, 90, 88, 78, 92, blank, and 89, the missing mark is filled using the integer average 86; the 78-mark row is filtered out.

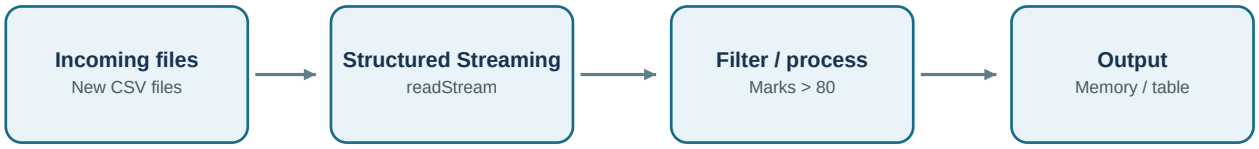
Experiment 9 – Real-Time Data Streaming

Aim: Simulate real-time data streaming and process incoming data using Apache Spark Structured Streaming in Databricks/cloud.

Platform Used: Databricks and Apache Spark Structured Streaming

1. Introduction

Structured Streaming processes continuously arriving data using Spark DataFrames. A file source can simulate incoming data by detecting new files added to a watched directory. This experiment uses a CSV file stream as a simple simulation.



2. Procedure

Step 1: Create a notebook — Create a Python notebook named Real_Time_Streaming.

Step 2: Define the schema — Define column names and data types for incoming student records.

Step 3: Configure a file stream — Use readStream to monitor a cloud-storage directory for new CSV files.

Step 4: Transform the stream — Filter incoming records where Marks is greater than 80.

Step 5: Start the query — Write results to a memory sink for a short demonstration.

Step 6: Add a new file — Upload a new CSV file to the watched input directory while the stream is running.

Step 7: Inspect and stop — Query the memory table and stop the streaming query.

3. Program / Commands

Define schema and read stream

```
from pyspark.sql.types import *

schema = StructType([
    StructField("Student_ID", IntegerType(), True),
    StructField("Name", StringType(), True),
    StructField("Department", StringType(), True),
    StructField("Marks", IntegerType(), True)
])

input_path = "/Volumes/<catalog>/<schema>/<volume>/incoming"

stream_df = (spark.readStream
    .schema(schema)
    .option("header", "true")
    .option("maxFilesPerTrigger", 1)
    .csv(input_path))
```

Filter and start streaming query

```
from pyspark.sql.functions import col

processed_stream = stream_df.filter(col("Marks") > 80)

query = (processed_stream.writeStream
    .format("memory")
    .queryName("student_results")
    .outputMode("append")
    .start())
```

Inspect and stop

```
query.processAllAvailable()
spark.sql("SELECT * FROM student_results").show()

query.stop()
```

4. Expected Output

New CSV files placed in the watched input directory are detected and processed. Records with Marks greater than 80 appear in the student_results memory table.

5. Result

Successfully simulated incoming data and processed it using Apache Spark Structured Streaming, applying a filter and displaying the resulting records.

6. Practical Notes

Replace the Volume path with a supported cloud-storage path. The memory sink is for demonstration and is not durable. Keep the query running while adding a new file. This is file-based streaming simulation, not a Kafka stream.

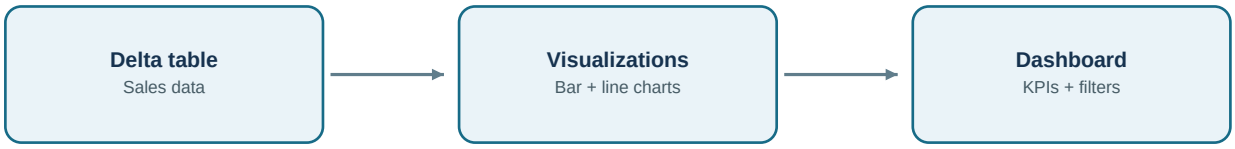
Experiment 10 – Sales Analytics Dashboard

Aim: Create meaningful visualizations such as bar charts and line charts and combine them into an interactive sales analytics dashboard using the processed data stored in the practical9_streaming_sales Delta table in Databricks.

Platform Used: Databricks, Delta Lake, Spark SQL, and dashboards

1. Introduction

A sales analytics dashboard combines charts and summary metrics to help users understand revenue, compare product performance, and monitor sales trends. Databricks can query Delta tables and present results through notebook visualizations and dashboards.



2. Procedure

Step 1: Open the notebook — Create a Python notebook named Sales_Analytics_Dashboard.

Step 2: Load the Delta table — Read the processed data from practical9_streaming_sales.

Step 3: Verify the schema — Check that the table contains Product, Sales, and Sale_Date columns. Adjust the code if the actual names differ.

Step 4: Create summary metrics — Calculate total sales, average sales, and number of records.

Step 5: Create a bar chart — Group sales by Product and visualize total sales for each product.

Step 6: Create a line chart — Group sales by Sale_Date and visualize daily sales trends.

Step 7: Build the dashboard — Add the summary and chart visualizations to a Databricks dashboard, arrange them, and save it.

3. Program / Commands

Load data and calculate summary

```
from pyspark.sql.functions import sum, avg, count, to_date

df = spark.table("practical9_streaming_sales")
df.printSchema()
display(df)

sales_summary = df.select(
    sum("Sales").alias("Total_Sales"),
    avg("Sales").alias("Average_Sales"),
    count("*").alias("Total_Records")
)
display(sales_summary)
```

Product-wise bar chart data

```
product_sales = (df.groupBy("Product")
    .agg(sum("Sales").alias("Total_Sales")))

display(product_sales)

# In the visualization settings:
# Chart type: Bar
# X-axis: Product
# Y-axis: Total_Sales
```

Daily line chart data

```
daily_sales = (df
```

```
    .withColumn("Sale_Date", to_date("Sale_Date"))
    .groupBy("Sale_Date")
    .agg(sum("Sales").alias("Total_Sales"))
    .orderBy("Sale_Date"))

display(daily_sales)

# In the visualization settings:
# Chart type: Line
# X-axis: Sale_Date
# Y-axis: Total_Sales
```

4. Expected Output

A sales summary displays total sales, average sales, and record count. A bar chart compares total sales by product. A line chart displays sales totals by date. The visualizations are combined in a dashboard.

5. Result

Successfully queried the practical9_streaming_sales Delta table, created sales summary metrics, generated product-wise bar and daily sales line charts, and assembled them into a sales analytics dashboard.

6. Practical Notes

This practical assumes the Delta table exists and has Product, Sales, and Sale_Date columns. Notebook display charts and a Databricks dashboard are separate steps: create the visualizations first, then add them to the dashboard. Dashboard menus vary by Databricks edition.